NUMPY

NumPy

NumPy: History

- Python language not initially designed for numerical computing
- Group called Matrix-SIG formed in 1995 to define an array computing package where Jim Fulton created the Numeric matrix package
- Numarray (large arrays) written as more flexible version of Numeric (small arrays)
- Travis Oliphant developed NumPy in 2005 as a unified package containing features from Numarray and Numeric
- NumPy released in 2006 as part of the SciPy package

NumPy: Motivation

- Python lists:
  - Are very flexible
  - Do not require uniform numerical types
  - Are very easy to modify (inserting or appending objects)
- However, flexibility often comes at the cost of performance, and lists are not the ideal object for numerical calculations



- Numpy is a Python module that defines a powerful n-dimensional array object that uses C and Fortran code behind the scenes to provide high performance
  - The downside of Numpy arrays is that they have a more rigid structure, and require a single numerical type (e.g. floating point values), but for a lot of scientific work, this is exactly what is needed.

NumPy: Basics

- NumPy's main object is the homogeneous multidimensional array
  - It is a table of elements of the same type (the type of the elements can be accesed by the attribute dtype)
    - Indexed by a tuple of positive integers
    - In Numpy dimensions are called axes
    - The number of axes is rank
- Example: the coordinates of a point in 3D space [1, 2, 1]
  - Array of rank 1, because it has one axis
  - That axis has a length of 3
- Example 2: [[1., 0., 0.],[0., 1., 2.]]
  - The array has rank 2 (it is 2-dimensional)
  - The first dimension (axis) has a length of 2
  - The second dimension has a length of 3

NumPy: Basics

- Numpy's array class is called ndarray
  - It is also known by the alias array
  - It is not the same as array.array, which only handles one-dimensional arrays and offers less functionality
- The more important attributes of an ndarray object are:
  - ndarray.ndim: number of axes (dimensions) of the array → rank
  - ndarray.shape: the dimensions of the array. This is a tuple of integers indicating the size of the array in each dimension
  - ndarray.size: the total number of elements of the array
  - ndarray.dtype: an object describing the type of the elements in the array
  - ndarray.data: the buffer containing the actual elements of the array

NumPy: array generation

- Using the array function
  - Create an array from a regular Python list or tuple
    - a = np.array([2,3,4])
  - Create an array from a sequences of sequences
    - b = np.array([(1.5,2,3), (4,5,6)])
  - The type of the array can also be explicitly specified at creation time
    - c = np.array([[1,2], [3,4]], dtype=complex)
- Using the matrix function
  - np.matrix( ) returns strictly two-dimensional arrays
- Create arrays with initial placeholder content
  - np.empty((2,3)): without initial values, they can vary
  - np.zeros((3,4)): array composed of zeros
  - np.ones((3,4)): array composed of ones
  - np.eye(3): identity matrix
  - np.random.rand(3,2): array composed of random float numbers

NumPy: array generation

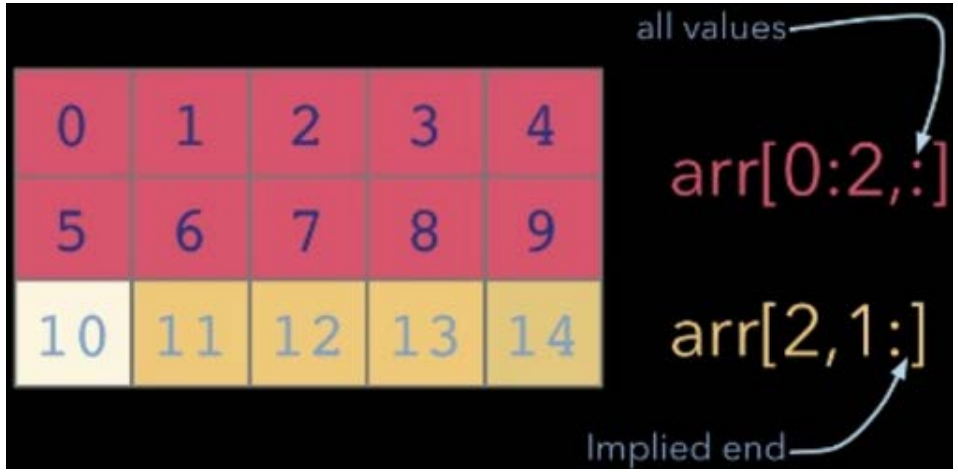
- To create sequences of numbers
  - arange function that can be used similarly to the built-in Python range function. It accepts float arguments
    - np.arange(10, 30, 5)
    - array([10, 15, 20, 25])
  - Linspace can be used to create linearly spaced values between and including limits
    - np.linspace(0, 2, 9) # 9 numbers from 0 to 2
    - array([0. , 0.25, 0.5 , 0.75, 1. , 1.25, 1.5 , 1.75, 2.])
  - Logspace: similar function can be used to create logarithmically spaced values

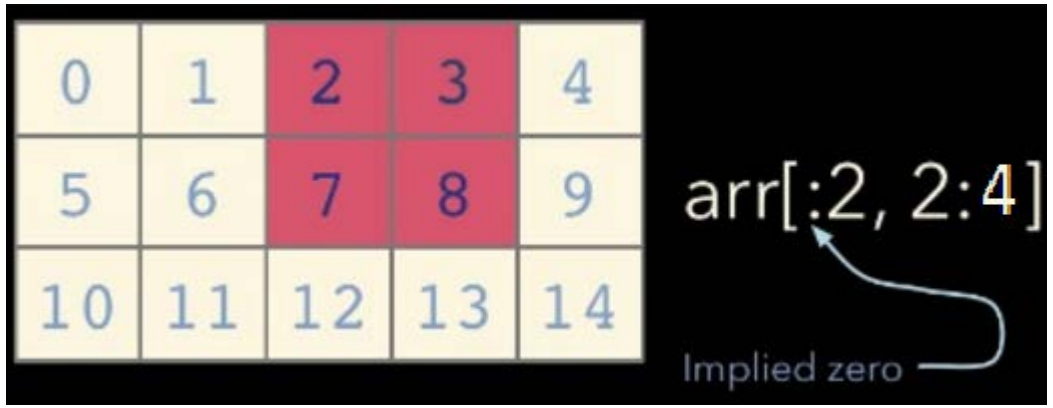
NumPy: Indexing and slicing

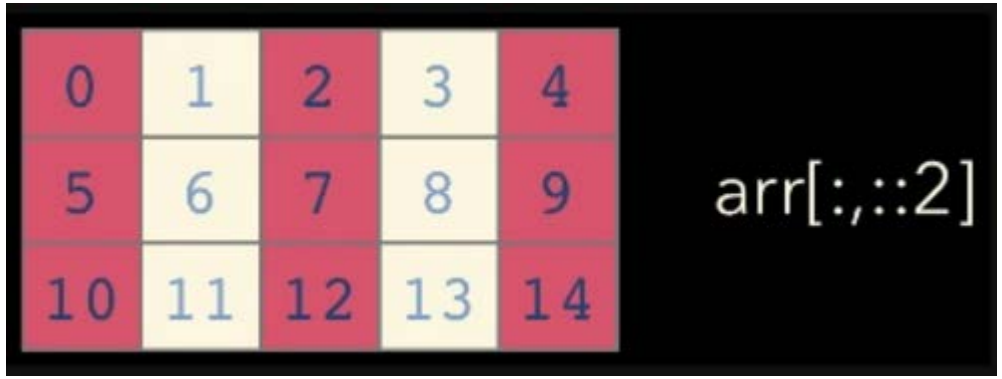
- Items in arrays can be accessed individually
  - x = np.array([9,8,7])
  - x[0] → 9
- Arrays can also be **sliced** by specifiying the start and end (not included)
  - y = np.arange(10)
  - y[0:5] or y[:5] → array([0, 1, 2, 3, 4])
- Optionally specifying a step:
  - y[0:10:2] or y[::2] → array([0, 2, 4, 6, 8])

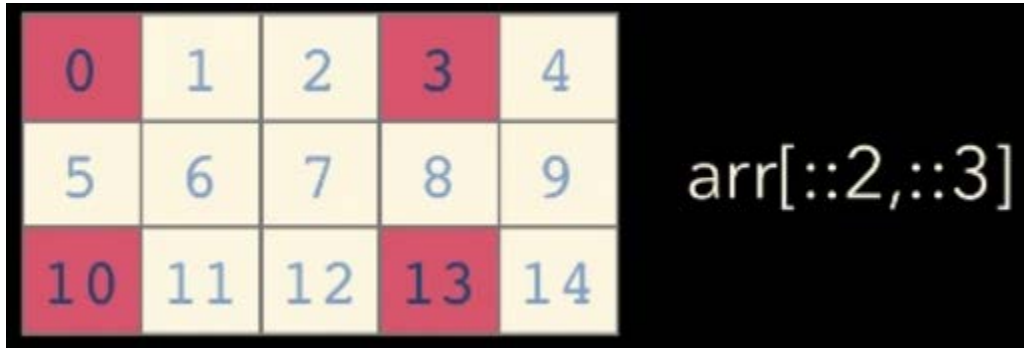
NumPy: Indexing and slicing

- Multi-dimensional arrays can be sliced differently along different dimensions









NumPy: Iterating

- Iterating over multidimensional arrays is done with respect to the first axis

```
>>> for row in b:
...     print(row)

[0 1 2 3]
[10 11 12 13]
[20 21 22 23]
[30 31 32 33]
[40 41 42 43]
```

- To perform an operation on each element in the array, one can use the flat attribute

```
>>> for element in b.flat:
...     print(element)
```

NumPy: Shape Manipulation

- Conversion an n-dimensional array in a 1-dimensional array, ravel

```
>>> a = np.floor(10*np.random.random((3,4)))
>>> a
array([[ 2.,  8.,  0.,  6.],
       [ 4.,  5.,  1.,  1.],
       [ 8.,  9.,  3.,  6.]])
>>> a.shape
(3, 4)
>>> a.ravel() # flatten the array
array([ 2.,  8.,  0.,  6.,  4.,  5.,  1.,  1.,  8.,  9.,  3.,  6.])
```

NumPy: Shape Manipulation

- Change the shape of the array (.T is the transpose of the array)

```
>>> a.shape = (6, 2)
>>> a.T
array([[ 2.,  0.,  4.,  1.,  8.,  3.],
       [ 8.,  6.,  5.,  1.,  9.,  6.]])
```

- Usage of the reshape function (returns its argument with a modified shape)
  - If a dimension is given as -1 in a reshaping operation, the other dimensions are automatically calculated

```
>>> b = np.arange(12).reshape(3,4)
>>> b
array([[ 0,  1,  2,  3],
       [ 4,  5,  6,  7],
       [ 8,  9, 10, 11]])
```

- Usage of the resize function (modifies the array itself)

```
>>> a.resize((2,6))
>>> a
array([[ 2.,  8.,  0.,  6.,  4.,  5.],
       [ 1.,  1.,  8.,  9.,  3.,  6.]])
```

NumPy: Stacking arrays

- Usage of the funcionts vstack and hstack

```
>>> a = np.floor(10*np.random.random((2,2)))
>>> a
array([[ 8.,  8.],
       [ 0.,  0.]])
>>> b = np.floor(10*np.random.random((2,2)))
>>> b
array([[ 1.,  8.],
       [ 0.,  4.]])
```

```
>>> np.vstack((a,b))
array([[ 8.,  8.],
       [ 0.,  0.],
       [ 1.,  8.],
       [ 0.,  4.]])
```

```
>>> np.hstack((a,b))
array([[ 8.,  8.,  1.,  8.],
       [ 0.,  0.,  0.,  4.]])
```

NumPy: basic operations

- Arithmetic operators on arrays apply *elementwise*. A new array is created and filled with the result.
  - This kind of functions are known as universal functions ufuncs, which dispatch to optimized C inner-loops based on array dtype.
- Numpy has many built-in ufuncs
  - Comparison: <, <=, ==, !=, >, >=
  - Arithmetic: +,-,*,/,reciprocal, square
  - Exponential: exp, expml, exp2, log, log10, loglp, log2, power, sqrt
  - Trigonometric: sin, cos, tan, acsin, arccos, atctan (hyperbolic: add an h at the end)
  - Bitwise: and, logical_xor, not, or
  - Predicates: isfinite, isinf, isnan, signbit
  - Other: abs, ceil, floor, mod, modf, round, sinc, sign, trunc

NumPy: basic operations examples

```
>>> a = np.array( [20,30,40,50] )
>>> b = np.arange( 4 )
>>> c = a-b
>>> c
    array([20, 29, 38, 47])
```

```
>>> a<35
    array([ True, True, False, False], dtype=bool)
```

NumPy: product

- Product operator * operates on each element of a NumPy array

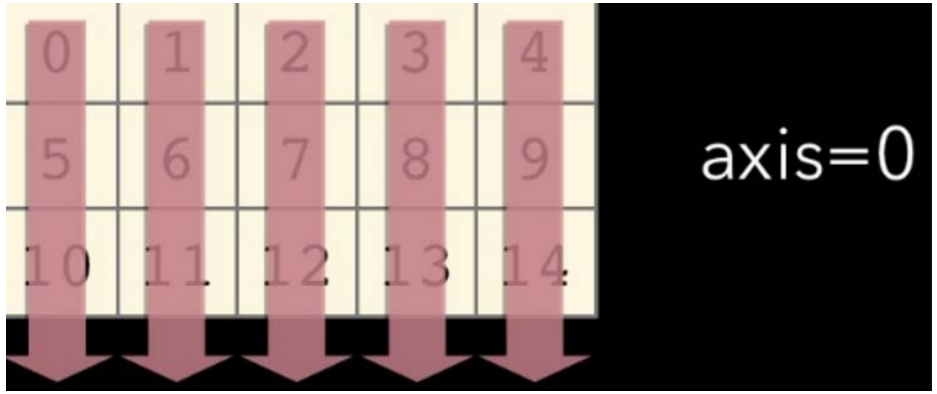
```
>>> A = np.array( [[1,1],
...               [0,1]] )
>>> B = np.array( [[2,0],
...               [3,4]] )
>>> A*B                         # elementwise product
array([[2, 0],
       [0, 4]])
```

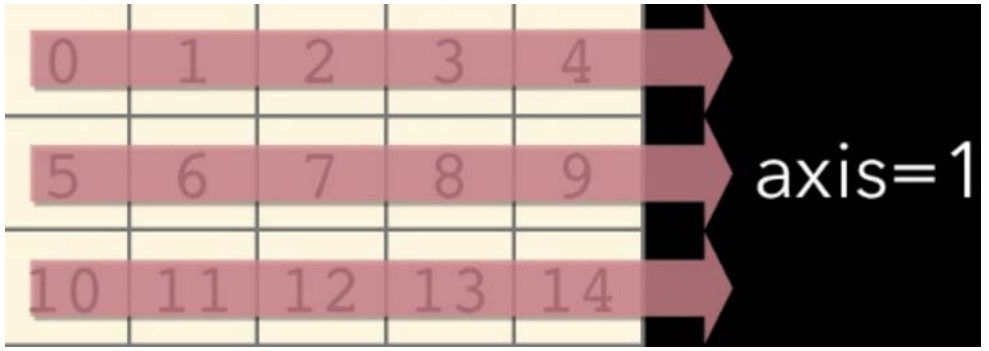
- Use np.dot( ) to calculate the matrix product

```
>>> A.dot(B)                    # matrix product
array([[5, 4],
       [3, 4]])
>>> np.dot(A, B)                # another matrix product
array([[5, 4],
       [3, 4]])
```

NumPy: basic operations (axis)

- The axis parameter applies operation to a specific axis of the array






```
a = np.array([[1,2,3,4],[5,6,7,8]])
# array([[1, 2, 3, 4],
#        [5, 6, 7, 8]])
```

```
a.sum(axis=0)
# array([ 6,  8, 10, 12])
```

```
a.sum(axis=1)
# array([10, 26])
```

Numpy: array methods

- Predicates: any, all
- Aggregations: mean, argmin, argmax, trace, cumsum, cumprod, sum, max, min, etc…
- Manipulation: argsort, transpose, etc…

NumPy: copying arrays

- NumPy arrays are mutable
  - To copy them use the copy method

```
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

b = a

b[0] = 99

b
# array([[99, 99, 99, 99],
#        [ 5,  6,  7,  8]])

a
# array([[99, 99, 99, 99],
#        [ 5,  6,  7,  8]])
```

```
a = np.array([[1, 2, 3, 4], [5, 6, 7, 8]])

c = a.copy()

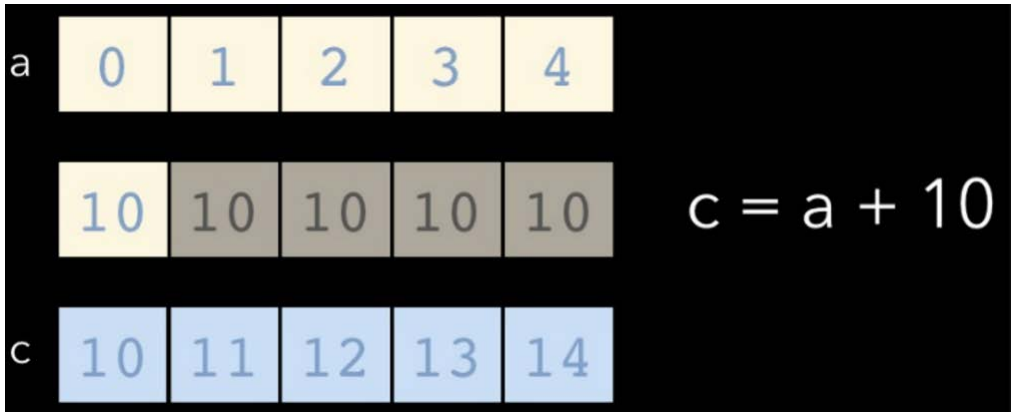
c[0] = 99

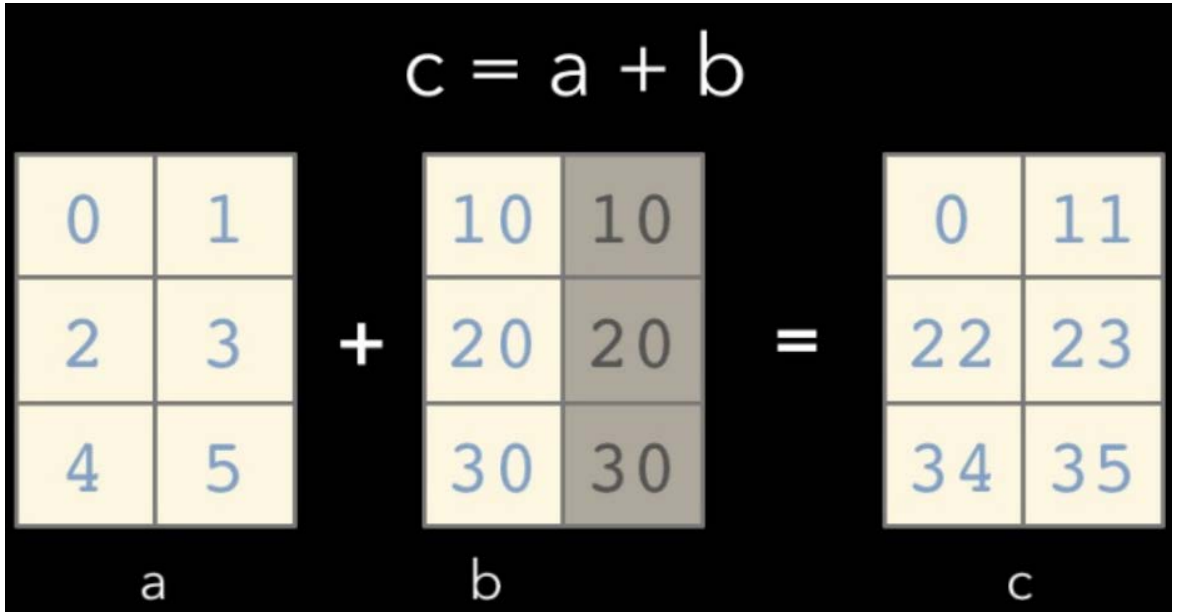
c
# array([[99, 99, 99, 99],
#        [ 5,  6,  7,  8]])

a
# array([[1, 2, 3, 4],
#        [5, 6, 7, 8]])
```

Broadcasting

- Broadcasting is used in NumPy to decide how to handle different shaped arrays
- Broadcasting makes your code more concise and fast
- Functions that support broadcasting are the universal functions (ufuncs)





Broadcasting

- Examples

```
a = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9], [10, 11, 12]])
# array([[ 1,  2,  3],
#        [ 4,  5,  6],
#        [ 7,  8,  9],
#        [10, 11, 12]])

b = np.array([10, 0, 10])
# array([10,  0, 10])

c = a + b
# array([[11,  2, 13],
#        [14,  5, 16],
#        [17,  8, 19],
#        [20, 11, 22]])
```

NumPy: Masking

- Accessing with a set of indexes

In [46]:

```
x = np.array([1,6,4,7,9,3,1,5,6,7,3,4,4,3])
x[[1,2,4,3,3,2]]
```

Out[46]:

```
array([6, 4, 9, 7, 7, 4])
```

- Using booleans

In [47]:

```
x[np.array([True, False, False, True, True, True, False, False, True, True, True, False, False, True])]
```

Out[47]:

```
array([1, 7, 9, 3, 6, 7, 3, 3])
```

- Or comparisons (even nested)

In [49]:

```
x[x > 3.4]
```

Out[49]:

```
array([6, 4, 7, 9, 5, 6, 7, 4, 4])
```

In [50]:

```
x[(x > 3.4) & (x < 5.5)]
```

Out[50]:

```
array([4, 5, 4, 4])
```

NumPy: Masking

- Using a mask to make assignments

```
y = np.array([1,6,4,7,9,3,1,5,6,7,3,4,4,3])
In [56]:
y[y > 5] = 0.
In [57]:
y
Out[57]:
array([1, 0, 4, 0, 0, 3, 1, 5, 0, 0, 3, 4, 4, 3])
```

NumPy: reading from text files

- numpy.loadtxt makes it easy to read in data from column-based data

```
In [41]:
%cat data/columns.txt
1995.00274 0.944444
1995.00548 -1.61111
1995.00821 -3.55556
1995.01095 -9.83333
1995.01369 -10.2222
1995.01643 -9.5
1995.01916 -10.2222
1995.02190 -6.61111
1995.02464 -2.94444
1995.02738 1.55556
1995.03012 0.277778
1995.03285 -1.44444
1995.03559 -3.61111
```

```
In [42]:
data = np.loadtxt('data/columns.txt') data
Out[42]:
array([[ 1.99500274e+03, 9.44444000e-01],
[1.99500548e+03, -1.61111000e+00],
[1.99500821e+03, -3.55556000e+00],
[1.99501095e+03, -9.83333000e+00],
[1.99501369e+03, -1.02222000e+01],
[1.99501643e+03, -9.50000000e+00],
[1.99501916e+03, -1.02222000e+01],
[1.99502190e+03, -6.61111000e+00],
[1.99502464e+03, -2.94444000e+00],
[1.99502738e+03, 1.55556000e+00],
[1.99503012e+03, 2.77778000e-01],
[1.99503285e+03, -1.44444000e+00],
[1.99503559e+03, -3.61111000e+00]])
```

NumPy: reading from text files

- Parameters for the loadtxt function (documentation for more details)
  - Comments: character or list of characters indicating a comment (default #)
  - Delimiter: string used to separate values (default whitespace)
  - Skiprows: number of rows to skip (default 0)
  - Usecols: list of columns to read (default is None returning all the columns)
  - Unpack: if True it returns individual columns

NumPy: reading from text files

- Example

```
In [43]:
date, temperature = np.loadtxt('data/columns.txt', unpack=True)
In [44]:
date
Out[44]:
array([ 1995.00274, 1995.00548, 1995.00821, 1995.01095, 1995.01369,
1995.01643, 1995.01916, 1995.0219 , 1995.02464, 1995.02738, 1995.03012,
1995.03285, 1995.03559])
In [45]:
temperature
Out[45]:
array([ 0.944444, -1.61111 , -3.55556 , -9.83333 , -10.2222 , -9.5 , -10.2222 , -
6.61111 , -2.94444 , 1.55556 , 0.277778, -1.44444 , -3.61111 ])
```

NumPy: Data I/O

- There are more functions
  - Fromfile
    - http://docs.scipy.org/doc/numpy/reference/generated/numpy.fromfile.html#numpy.fromfile
  - Fromstring
    - http://docs.scipy.org/doc/numpy/reference/generated/numpy.fromstring.html#numpy.fromstring
  - Genfromtxt
    - http://docs.scipy.org/doc/numpy/reference/generated/numpy.genfromtxt.html#numpy.genfromtxt
  - Load
    - http://docs.scipy.org/doc/numpy/reference/generated/numpy.load.html#numpy.load
  - Save
    - http://docs.scipy.org/doc/numpy/reference/generated/numpy.save.html#numpy.save
  - Savetxt
    - http://docs.scipy.org/doc/numpy/reference/generated/numpy.savetxt.html#numpy.savetxt

Numpy: more information

- Tutorial for more information
  - https://docs.scipy.org/doc/numpy-dev/user/quickstart.html